

Apply filters to SQL queries

Project description

Using SQL queries and filtering techniques, I investigated security-related data from employee and login records. I retrieved failed login attempts made after business hours, identified login attempts on specific dates, and analyzed login activity originating outside of Mexico. I also filtered employee records by department to locate Marketing employees, employees in the Finance or Sales departments, and employees who were not in the Information Technology department.

Retrieve after hours failed login attempts

```
MariaDB [organization]> clear
MariaDB [organization]> SELECT *
  -> FROM log_in_attempts
  -> WHERE login_time > '18:00' AND success = FALSE;
```

event_id	username	login_date	login_time	country	ip_address	success
2	apatel	2022-05-10	20:27:27	CAN	192.168.205.12	0
18	pwashing	2022-05-11	19:28:50	US	192.168.66.142	0
20	tshah	2022-05-12	18:56:36	MEXICO	192.168.109.50	0
28	aestrada	2022-05-09	19:28:12	MEXICO	192.168.27.57	0
34	drosas	2022-05-11	21:02:04	US	192.168.45.93	0
42	cgriffin	2022-05-09	23:04:05	US	192.168.4.157	0
52	cjackson	2022-05-10	22:07:07	CAN	192.168.58.57	0
69	wjaffrey	2022-05-11	19:55:15	USA	192.168.100.17	0
82	abernard	2022-05-12	23:38:46	MEX	192.168.234.49	0
87	apatel	2022-05-08	22:38:31	CANADA	192.168.132.153	0
96	ivelasco	2022-05-09	22:36:36	CAN	192.168.84.194	0
104	asundara	2022-05-11	18:38:07	US	192.168.96.200	0
107	bisles	2022-05-12	20:25:57	USA	192.168.116.187	0
111	aestrada	2022-05-10	22:00:26	MEXICO	192.168.76.27	0
127	abellmas	2022-05-09	21:20:51	CANADA	192.168.70.122	0
131	bisles	2022-05-09	20:03:55	US	192.168.113.171	0
155	cgriffin	2022-05-12	22:18:42	USA	192.168.236.176	0
160	jclark	2022-05-10	20:49:00	CANADA	192.168.214.49	0
199	yappiah	2022-05-11	19:34:48	MEXICO	192.168.44.232	0

```
19 rows in set (0.030 sec)

MariaDB [organization]>
```

A potential security incident was involving login attempts made after business hours (after 18:00). Therefore, all failed login attempts during this time period need to be investigated.

I started writing a SQL query to filter records from the `log_in_attempts` table. It selects all data from the table, and then applies a `WHERE` clause with an `AND` operator to narrow down the results

The first condition `login_time > '18:00'` filters for login attempts that occurred after 18:00. The second condition `success = FALSE` ensures that only unsuccessful login attempts are included in the results. Together, these conditions return all failed login attempts that occurred after business hours.

Retrieve login attempts on specific dates

```
MariaDB [organization]> SELECT *  
-> FROM log_in_attempts  
-> WHERE login_date = '2022-05-09' OR login_date = '2022-05-08';
```

event_id	username	login_date	login_time	country	ip_address	success
1	jrafael	2022-05-09	04:56:27	CAN	192.168.243.140	1
3	dkot	2022-05-09	06:47:41	USA	192.168.151.162	1
4	dkot	2022-05-08	02:00:39	USA	192.168.178.71	0

A suspicious event was detected on 2022-05-09, therefore all login activity from that date and the previous day must be reviewed.

I created a SQL query to filter records from the `log_in_attempts` table for specific dates. The query selects all data from the table, and then uses a `WHERE` clause with an `OR` operator to return results from either of the two relevant days.

The first condition, `login_date = '2022-05-09'`, retrieves login attempts that occurred on the day of the incident. The second condition, `login_date = '2022-05-08'`, captures activity from the day before.

Retrieve login attempts outside of Mexico

```
MariaDB [organization]> SELECT *  
-> FROM log_in_attempts  
-> WHERE NOT country LIKE 'MEX%';
```

event_id	username	login_date	login_time	country	ip_address	success
1	jrafael	2022-05-09	04:56:27	CAN	192.168.243.140	1
2	apatel	2022-05-10	20:27:27	CAN	192.168.205.12	0
3	dkot	2022-05-09	06:47:41	USA	192.168.151.162	1

After reviewing the organization's login attempt data, I identified a potential issue involving login activity originating from outside of Mexico. These attempts need further investigation.

I created a SQL query using the `log_in_attempts` table. The query begins by selecting all records, and then applies a `WHERE` clause with the `NOT` operator to exclude login attempts from Mexico.

To handle ways that Mexico is recorded in the dataset such as `MEX` and `MEXICO`, I used the `LIKE` operator with the pattern `MEX%`. The `%` allows the query to match any value that starts with MEX, no matter what follows.

Retrieve employees in Marketing

```
MariaDB [organization]> SELECT *  
  -> FROM employees  
  -> WHERE department = 'Marketing' AND office LIKE 'East%';  
+-----+-----+-----+-----+-----+  
| employee_id | device_id | username | department | office |  
+-----+-----+-----+-----+-----+  
|      1000 | a320b137c219 | elarson | Marketing | East-170 |  
|      1052 | a192b174c940 | jdarosa | Marketing | East-195 |  
|      1075 | x573y883z772 | fbautist | Marketing | East-267 |  
|      1088 | k865l965m233 | rgosh | Marketing | East-157 |  
|      1103 | NULL | randerss | Marketing | East-460 |  
|      1156 | a184b775c707 | dellery | Marketing | East-417 |  
|      1163 | h679i515j339 | cwilliam | Marketing | East-216 |  
+-----+-----+-----+-----+-----+  
7 rows in set (0.001 sec)  
  
MariaDB [organization]> 
```

My team wants to update computers for employees in the Marketing department, so I had to identify which employee machines are affected.

I wrote a SQL query using the `employees` table to filter for the relevant records. The query selects all data from the table, and applies a `WHERE` clause with an `AND` operator to narrow the results.

The first condition, `department = 'Marketing'`, filters for employees who are working in the Marketing department. The second condition, `office LIKE 'East%'`, filters for employees located in the East building. The `%` wildcard is used to match any additional characters following “East,” which will capture employees in the East building.

Retrieve employees in Finance or Sales

```
MariaDB [organization]> SELECT *  
-> FROM employees  
-> WHERE department = 'Finance' OR department = 'Sales';
```

employee_id	device_id	username	department	office
1003	d394e816f943	sgilmore	Finance	South-153
1007	h174i497j413	wjaffrey	Finance	North-406
1008	i858j583k571	abernard	Finance	South-170

The machines for employees in the Finance and Sales departments also require a security update. I need to retrieve the relevant employee information from both groups.

I created a SQL query using the `employees` table, and applied a `WHERE` clause with the `OR` operator to filter the results. It ensures the employees from either department are included in the output.

The first condition, `department = 'Finance'`, selects employees in the Finance department. The second condition, `department = 'Sales'`, selects employees in the Sales department. Using `OR` allows the query to return records that match either condition, ensuring all employees in both departments are included for the update.

Retrieve all employees not in IT

```
MariaDB [organization]> SELECT *  
-> FROM employees  
-> WHERE NOT department = 'Information Technology';
```

employee_id	device_id	username	department	office
1000	a320b137c219	elarson	Marketing	East-170
1001	b239c825d303	bmoreno	Marketing	Central-276
1002	c116d593e558	tshah	Human Resources	North-434

My team needs to perform a final security update for employees who are not part of the Information Technology department. I first needed to identify the relevant employee machines.

I wrote a SQL query using the `employees` table, and applied a `WHERE` clause with the `NOT` operator to exclude employees in the Information Technology department.

The condition `department = 'Information Technology'` identifies employees in that department, and using `NOT` ensures these records are excluded from the results. It keeps only

employees from all other departments, which are included in the output for the required security update.

Summary

I used SQL filters to retrieve specific information about login attempts and employee machines by working with two tables: `log_in_attempts` and `employees`.

Across the queries, I applied the `AND`, `OR`, and `NOT` operators to filter results based on the requirements of each investigation. These operators helped combine or exclude conditions to isolate the exact records that I needed.

Moreover, I used the `LIKE` operator along with the percentage sign `%` wildcard to filter for patterns, which allows for more flexible filtering of the database.